

Lab 2

ELECENG 3EY4

Alaa Abdelsalam – abdel74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

February 1, 2024

Table of Contents

Objective 5: Workspace Setup and Package Installation	2
Question 1	2
Question 2	2
Question 3	3
Question 4	3
Question 5	3
Objective 6: ROS Basics Practical Example	4
Question 6	4
Question 7	4
Question 8	5

Objective 5: Workspace Setup and Package Installation

Question 1

When creating a ROS package, it is required to have dependencies “std_msgs,” “roscpp,” and “rospy.” It is required to have these dependencies as they specify the external libraries and tools that the package depends on. The “std_msgs” dependency provides a set of standard message types. This includes common data structures including integers, floats, strings, and more. The “roscpp” dependency is a C++ client library. This allows for the creation of ROS nodes in C++. This dependency also allows us to interact with the ROS system by utilizing C++ code. The “rospy” dependency is a Python client library. This allows for the creation of ROS nodes in Python. This dependency also allows us to interact with the ROS system by utilizing Python code.

Question 2

“sudo”: This stands for “superuser do.” This is a command that grants “superuser do” permissions, which allows the user to run a command as the superuser. This will allow us to make installations.

“apt-get”: “apt” stands for “advanced package tool.” This line is used to handle installing, updating, upgrading, and removing packages.

“install”: This is the argument that is passed to “apt-get,” which tells it that it wants to perform an installation of a specified package or library.

“ros-melodic-serial”: This line shows the name of a VESC dependency, which is called “serial” for the Melodic version. This is one of the packages being installed in the command.

“ros-melodic-ackermann-msgs”: This line shows the name of another VESC dependency, which is called “Ackermann-msgs” for the Melodic version. This is another package being installed in the command.

“ros-melodic-rplidar-ros”: This line shows the VESC dependency that is for the lidar sensor, which is designed for ROS Melodic. This is another package being installed in the command.

“ros-melodic-realsense2-camera”: This line shows the VESC dependency for the camera, which is designed for ROS Melodic.

“libusb-dev”: This line shows the development library for the USB protocol. This allows for interaction with USB devices. This is important because the joystick driver is dependent on this. This is one of the libraries being installed in the command.

“libspnav-dev”: This line shows the development library for the SpaceNavigator, which is for a motion controller. This is important because the joystick driver is dependent on this. This is another library being installed in the command.

Question 3

To remove the Wiimote driver, the “`sudo rm -r wiimote`” command was typed in the terminal. This command was typed in the path `~/catkin_ws/src/joystick_drivers`. This command uses the “`sudo`” and “`rm -r`” commands. The “`sudo`” command grants us permission to run the remove command (“`rm -r`”) to delete the driver as the superuser. The “`rm -r`” command recursively removes a directory and its contents. The “`-r`” denotes the recursion of deleting all the files within the Wiimote driver directory, and then deletes the directory. Following the “`rm -r`” command, “`wiimote`” is also typed. This represents the directory that we are removing. By using the “`sudo rm -r wiimote`” command, we are able to remove the Wiimote driver directory and its contents. This explains why we used this command.

Question 4

The command used to build the packages is “`catkin_make`.” This command was typed in the path `~/catkin_ws`. Building packages in ROS is a very important step to compile source code and ensure the code is integrated into the ROS environment to be ready for system execution. It updates the ROS workplace to support robotic applications. It is required that you build the packages installed earlier because they produce the executable files that can be run in the ROS environment.

Question 5

In the last step of Objective 5, it is required to edit the `.bashrc` file. When editing this file in the last step of Objective 5, we add the line “`source ~/catkin_ws/devel/setup.bash`” to the end of the file. This line serves the purpose of sourcing the setup files produced during the build process by the command “`catkin_make`.” By incorporating this line at the end of the `.bashrc` file, it configures the Linux shell so it will recognize the ROS workspace. When we add the source line at the end of the `.bashrc` file, it guarantees that each time a new terminal is opened, the environment setup will be consistent. This signifies that it is automating the process of setting up the ROS environment variables every time a new terminal is opened. This indicates that manually typing in the line “`source ~/catkin_ws/devel/setup.bash`” every time a new terminal is opened, will not be required. This removes the requirement to manually source the setup files when opening a new terminal. By adding “`source ~/catkin_ws/devel/setup.bash`” to the `.bashrc` file, it ensures that the environment setup is completed before proceeding to work with ROS, without having to manually type the line “`source ~/catkin_ws/devel/setup.bash`.”

Objective 6: ROS Basics Practical Example

Question 6

Both entries “/rosout” and “/rosout_agg” are used to help communicate and monitor what is occurring in the ROS system by giving updates regarding any issues from nodes. The purpose of the entry “/rosout” is for logging. This is a communication channel that publishes log information in the ROS system. Nodes can publish log messages that include information regarding statuses, warnings, errors, and debugging. This also allows for monitoring of the ROS during run time. The purpose of the entry “/rosout_agg” is for message aggregation. This means that it aggregates log messages from various nodes in the ROS system. Furthermore, this means that it provides a consolidated view of all log messages by grouping similar log entries depending on their content/information. This allows us to view an organized and concise summary of the logged messages from different nodes. By doing this, it also removes the clutter of separate log messages.

Question 7

“rostopic pub”: “pub” stands for “publish.” This part of the command publishes the data to a topic.

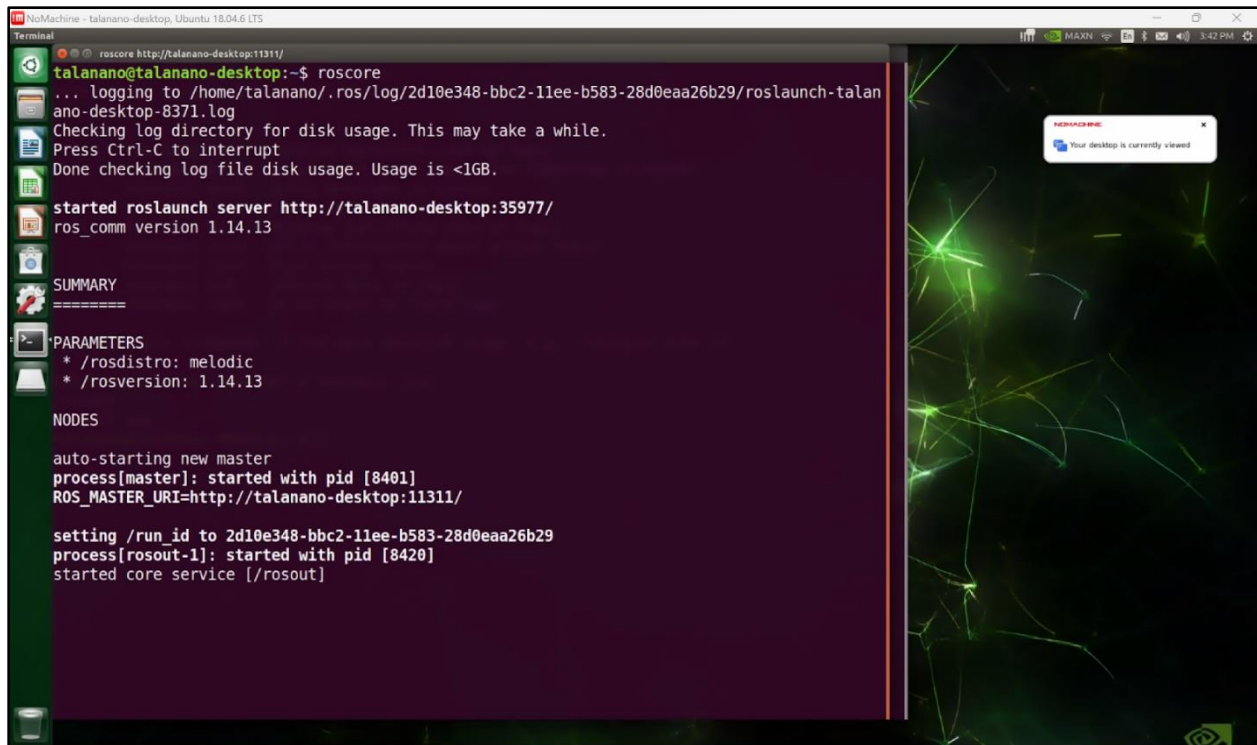
“/hello”: This part of the command is the topic name to which the data (or message) will be published.

“std_msgs/String”: This part of the command specifies the message type, which is identified as “String.” This means that the data being published will be a string.

“Hello Robot”: This part of the command is the actual data or message that is being published to the “/hello” topic. This means that the message is the string “Hello Robot.”

Question 8

In Objective 6, there were a few main things that we did and learned. In this objective, we utilized and learned about different ROS tools. To begin, we started up the ROS Master using the “roscore” command. This terminal was not closed as it was required to keep it running in order for ROS nodes to communicate. This is shown in Figure 1 below. Then, the “rostopic” command was used to display and view the different options of information that can be printed about ROS topics. It listed out the various commands that can be used to print information about ROS topics. Then, the “rostopic list” command was used to list the active topics. This gave us an output of entries called “/rosout” and “/rosout_agg.” These are shown in Figure 2 below. Next, we published a message. To do this, a new terminal was opened and the command “rostopic pub /hello std_msgs/String “Hello Robot” was used. This command published the string “Hello Robot” to the “/hello” topic. After executing this command, a new topic called “/hello” was created. This was seen by returning to the previous terminal and using the “rostopic list” command again. This showed that there were now three topics including “/hello,” “/rosout,” and “/rosout_agg.” To view the message published to the topic “/hello,” the “rostopic echo /hello” command was used. This gave an output of “data: Hello Robot.” We also viewed the node that published the message, which was determined by using the “rostopic list” command. This node that published the message was created by ROS since we asked “rostopic” to publish the “/hello” topic. This node was called “/rostopic_9234_1706215616780.” Then, we used the “rostopic info /rostopic_9234_1706215616780” command to see the details of the node. The output of this command provided information regarding the publications, subscriptions, and services for the node. Similarly, we used the “rostopic info /hello” command to see the details of the topic “/hello.” The output of this command provided information regarding the publishers and subscribers of the topic. These are shown in Figure 3 below. These are the various steps we conducted and learned in Objective 6.



```
talanano@talanano-desktop:~$ roscore
... logging to /home/talanano/.ros/log/2d10e348-bbc2-11ee-b583-28d0eaa26b29/roslaunch-talanano-desktop-8371.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://talanano-desktop:35977/
ros_comm version 1.14.13

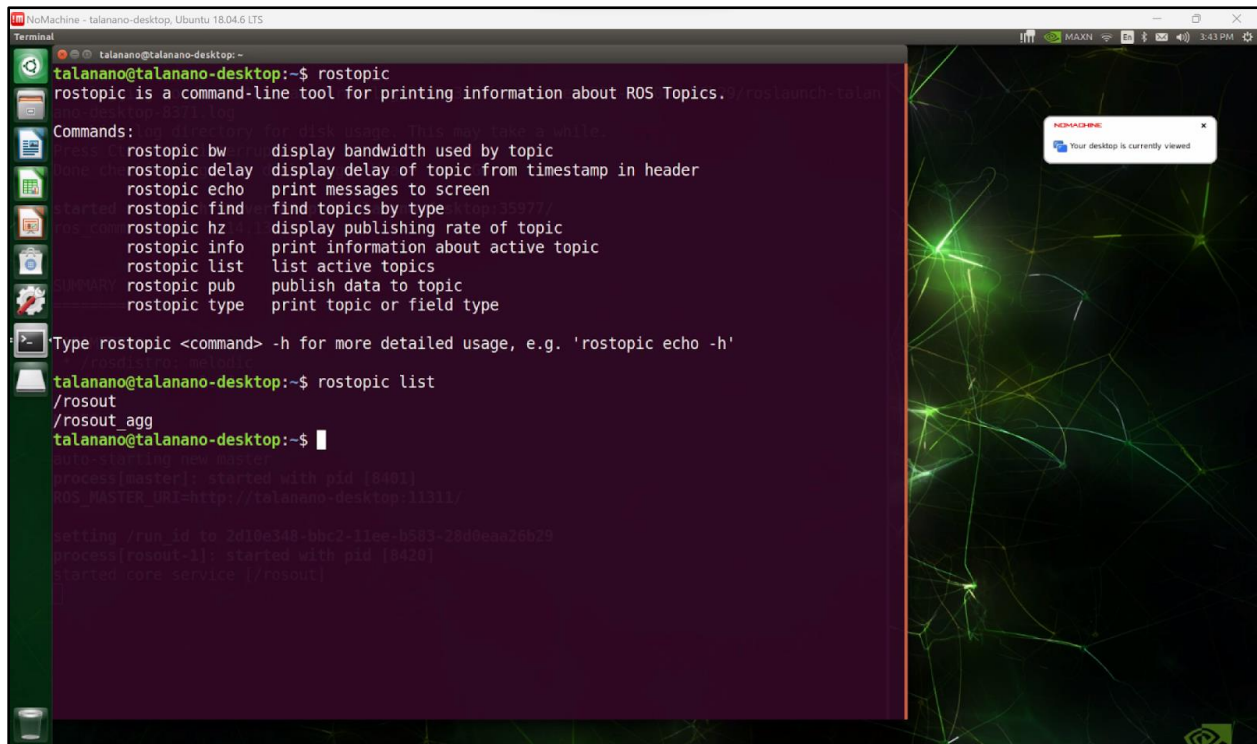
SUMMARY
=====
PARAMETERS
* /rostdistro: melodic
* /rosversion: 1.14.13

NODES

auto-starting new master
process[master]: started with pid [8401]
ROS_MASTER_URI=http://talanano-desktop:11311/

setting /run_id to 2d10e348-bbc2-11ee-b583-28d0eaa26b29
process[rosout-1]: started with pid [8420]
started core service [/rosout]
```

Figure 1. Objective 6: “roscore” Command and Output



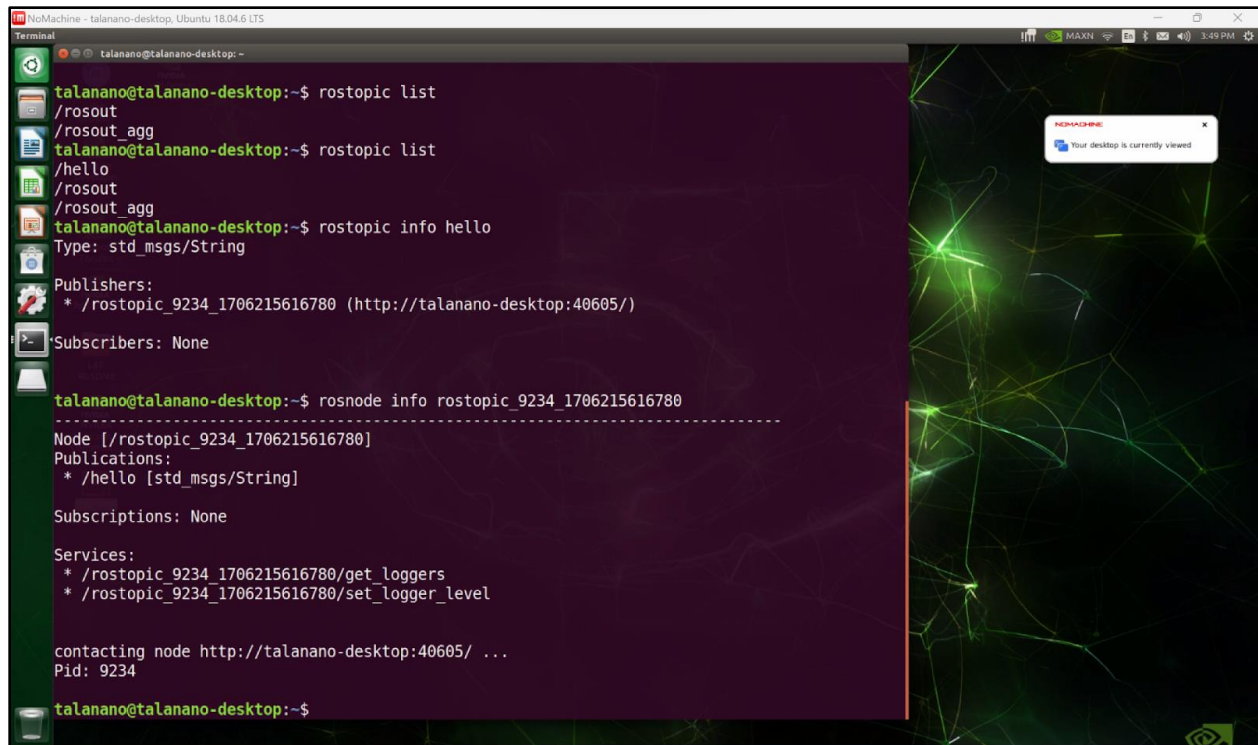
```
talanano@talanano-desktop:~$ rostopic
rostopic is a command-line tool for printing information about ROS Topics.

Commands:
rostopic bw      display bandwidth used by topic
rostopic delay   display delay of topic from timestamp in header
rostopic echo    print messages to screen
rostopic find    find topics by type
rostopic hz      display publishing rate of topic
rostopic info    print information about active topic
rostopic list    list active topics
rostopic pub     publish data to topic
rostopic type    print topic or field type

Type rostopic <command> -h for more detailed usage, e.g. 'rostopic echo -h'

talanano@talanano-desktop:~$ rostopic list
/rosout
/rosout_agg
talanano@talanano-desktop:~$
```

Figure 2. Objective 6: “rostopic” and “rostopic list” Commands and Outputs



```
talanano@talanano-desktop:~$ rostopic list
/rosout
/rosout_agg
talanano@talanano-desktop:~$ rostopic list
/hello
/rosout
/rosout_agg
talanano@talanano-desktop:~$ rostopic info hello
Type: std_msgs/String

Publishers:
* /rostopic_9234_1706215616780 (http://talanano-desktop:40605/)

Subscribers: None

talanano@talanano-desktop:~$ rosnode info rostopic_9234_1706215616780
-----
Node [/rostopic_9234_1706215616780]
Publications:
* /hello [std_msgs/String]

Subscriptions: None

Services:
* /rostopic_9234_1706215616780/get_loggers
* /rostopic_9234_1706215616780/set_logger_level

contacting node http://talanano-desktop:40605/ ...
Pid: 9234

talanano@talanano-desktop:~$
```

Figure 3. Objective 6: “*rosnode info /rostopic_9234_1706215616780*” and “*rostopic info /hello*” Commands and Outputs